

Title: software company

Aim

Different categories of employees work in a software company — Regular Employees, Contract Employees, and Vendors — and their payroll differs. Using an **interface**, find and display the monthly salary details of an employee based on the Employee ID entered:

- Regular Employee: Basic Pay 25000, HRA 15000, T.A 5000
- Contract Employee: Basic Pay 12000, T.A 3000, no HRA
- Vendor: no monthly salary details

TestCase 1

```
Input:  Enter Employee Id: R101
Output: Salary Details:
        Basic Pay: 25000
        HRA: 15000
        T.A: 5000
        Total Amount: 45000
```

System Requirements

Hardware

```
Standard PC / Laptop
Intel Core i3 / i5 processor or equivalent
Minimum 4 GB RAM
Minimum 500 MB free storage
```

Software

```
Operating System : Windows / Linux / macOS
Java Development Kit : JDK 17 or JDK 21
Text Editor or IDE : Eclipse, IntelliJ IDEA, NetBeans, or VS Code
Command Prompt / Terminal
```

Source Code

```
package inheritance;
```

```
import java.util.Scanner;

// Interface for all company categories.
interface Employee {
    void salaryDetails();
}

// Regular employees get Basic Pay, HRA and T.A.
class RegularEmployee implements Employee {
    @Override
    public void salaryDetails() {
        int basicPay = 25000;
        int hra = 15000;
        int ta = 5000;
        int totalAmount = basicPay + hra + ta;

        System.out.println("Salary Details:");
        System.out.println("Basic Pay: " + basicPay);
        System.out.println("HRA: " + hra);
        System.out.println("T.A: " + ta);
        System.out.println("Total Amount: " + totalAmount);
    }
}

// Contract employees get Basic Pay and T.A. There is no HRA.
```

```
class ContractEmployee implements Employee {  
    @Override  
    public void salaryDetails() {  
        int basicPay = 12000;  
        int ta = 3000;  
        int totalAmount = basicPay + ta;  
  
        System.out.println("Salary Details:");  
        System.out.println("Basic Pay: " + basicPay);  
        System.out.println("HRA: 0");  
        System.out.println("T.A: " + ta);  
        System.out.println("Total Amount: " + totalAmount);  
    }  
}
```

// Vendors do not have a monthly employee salary.

```
class Vendor implements Employee {  
    @Override  
    public void salaryDetails() {  
        System.out.println("Vendor has no monthly salary  
details.");  
    }  
}
```

```
public class SoftwareCompany {
```

```
public static void main(String[] args) {  
    Scanner scanner = new Scanner(System.in);  
  
    System.out.print("Enter Employee Id: ");  
    String employeeId = scanner.nextLine().toUpperCase();  
  
    Employee employee;  
  
    switch (employeeId.charAt(0)) {  
        case 'R':  
            employee = new RegularEmployee();  
            break;  
        case 'C':  
            employee = new ContractEmployee();  
            break;  
        case 'V':  
            employee = new Vendor();  
            break;  
        default:  
            System.out.println("Invalid Employee Id.");  
            scanner.close();  
            return;  
    }  
  
    employee.salaryDetails();  
}
```

```
        scanner.close();
    }
}
```

An **interface** is a contract that declares *what* a class must do, without saying *how*. It contains method signatures with no body (here, `void salaryDetails();`). Any class that **implements** the interface must provide its own body for that method.

```
Defining an interface      -> interface Employee { void salaryDetails(); }
Implementing an interface -> class RegularEmployee implements Employee { ...
}
Accessing through an      -> Employee employee = new RegularEmployee();
    interface reference    employee.salaryDetails();
                           (the reference type is the INTERFACE,
                           the object type is the IMPLEMENTING class)
```

Three unrelated classes — `RegularEmployee`, `ContractEmployee`, `Vendor` — all implement the same `Employee` interface. This lets `main()` treat all of them the same way through one reference type (`Employee employee`), and Java calls the correct class's version of `salaryDetails()` at runtime — this is **runtime polymorphism through an interface**.

This program does not extend one interface from another (extends between interfaces), it only demonstrates **defining, implementing, and accessing through an interface reference**.

Exception

This program does **not use try-catch or throw any exception class**. Instead, it does **manual input validation**

```
switch (employeeId.charAt(0)) {
    ...
    default:
        System.out.println("Invalid Employee Id.");
        scanner.close();
        return;
}
```

If the Employee ID does not start with R, C, or V, the program prints "Invalid Employee Id." and exits gracefully using `return` — instead of letting an exception occur.

Potential hidden exception: if the user enters an **empty string** (just presses Enter), `employeeId.charAt(0)` will throw a `StringIndexOutOfBoundsException` (an **unchecked runtime exception**), because there is no character at index 0 of an empty string. The program does not currently guard against this case.

Exception (potential, unhandled)	When it could occur	Type
<code>StringIndexOutOfBoundsException</code>	User enters an empty Employee ID	Unchecked (<code>RuntimeException</code>)

2. Step-wise Execution

1. Scanner object is created to read input.
 2. User is prompted: "Enter Employee Id: " (e.g., "R101").
 3. Input is converted to uppercase and stored in `employeeId`.
 4. `employee` reference variable is declared with `INTERFACE` type `Employee`.
 5. A switch checks the FIRST character of `employeeId`:
 - 'R' -> `employee = new RegularEmployee()`
 - 'C' -> `employee = new ContractEmployee()`
 - 'V' -> `employee = new Vendor()`
 - anything else -> print "Invalid Employee Id.", close Scanner, exit
 6. `employee.salaryDetails()` is called.
 - > Java looks at the ACTUAL object type at runtime and runs THAT class's implementation (dynamic dispatch through the interface reference).
 7. IF object is `RegularEmployee`:
 - `basicPay = 25000, hra = 15000, ta = 5000`
 - `totalAmount = 25000 + 15000 + 5000 = 45000`
 - Prints: Basic Pay: 25000, HRA: 15000, T.A: 5000, Total Amount: 45000
 8. IF object is `ContractEmployee`:
 - `basicPay = 12000, ta = 3000 (no HRA)`
 - `totalAmount = 12000 + 3000 = 15000`
 - Prints: Basic Pay: 12000, HRA: 0, T.A: 3000, Total Amount: 15000
 9. IF object is `Vendor`:
 - Prints: "Vendor has no monthly salary details."
 10. Scanner is closed.
-

3. Algorithm

START

1. Create Scanner object
2. Read Employee Id from user, convert to uppercase
3. Declare reference: `Employee employee`
4. SWITCH on first character of Employee Id:
 - CASE 'R': `employee = new RegularEmployee()`
 - CASE 'C': `employee = new ContractEmployee()`
 - CASE 'V': `employee = new Vendor()`
 - DEFAULT: print "Invalid Employee Id.", close Scanner, END
5. CALL `employee.salaryDetails()`
 - IF object is `RegularEmployee`:

```

        basicPay = 25000
        hra = 15000
        ta = 5000
        totalAmount = basicPay + hra + ta
        PRINT basicPay, hra, ta, totalAmount
    IF object is ContractEmployee:
        basicPay = 12000
        ta = 3000
        totalAmount = basicPay + ta
        PRINT basicPay, "HRA: 0", ta, totalAmount
    IF object is Vendor:
        PRINT "Vendor has no monthly salary details."
6. Close Scanner
STOP

```

4. UML Class Diagram



```

+-----+
| + main(String[] args) : void |
+-----+
| creates one of the three classes,
| but stores it using the interface
| reference: Employee employee
v
Employee <-- (reference type used to call salaryDetails())

```

```

+-----+
| Scanner |
+-----+
| + nextLine() : String |
| + close() : void |
+-----+

```

Key points shown:

- Employee is an **interface** (marked <<interface>>), declaring only the method signature.
- RegularEmployee, ContractEmployee, Vendor **each implement** Employee (hollow-triangle arrows) and give their own body for salaryDetails().
- SoftwareCompany.main() creates the correct implementing class but refers to it using the **interface type** Employee — this is **accessing an implementation through an interface reference**.
- At runtime, calling employee.salaryDetails() executes the version belonging to the *actual* object — this is **polymorphism via interfaces**.

Compilation and Execution

Folder structure:

```

project/
├── inheritance/
│   └── SoftwareCompany.java

```

Compile:

```
javac inheritance/SoftwareCompany.java
```

Run:

```
java inheritance.SoftwareCompany
```

Sample Output

```

Enter Employee Id: R101
Salary Details:
Basic Pay: 25000
HRA: 15000

```

```
T.A: 5000
Total Amount: 45000

Enter Employee Id: C205
Salary Details:
Basic Pay: 12000
HRA: 0
T.A: 3000
Total Amount: 15000

Enter Employee Id: V301
Vendor has no monthly salary details.

Enter Employee Id: X999
Invalid Employee Id.
```

Output:

```
PS C:\Users\Admin\OneDrive\Documents\basicjava> c:: cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'c:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.softwarecompany'
Enter Employee Id: c232346
Salary Details:
Basic Pay: 12000
HRA: 0
T.A: 3000
Total Amount: 15000
PS C:\Users\Admin\OneDrive\Documents\basicjava> c:: cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'c:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.softwarecompany'
Enter Employee Id: r324252e
Salary Details:
Basic Pay: 25000
HRA: 15000
T.A: 5000
Total Amount: 45000
PS C:\Users\Admin\OneDrive\Documents\basicjava> c:: cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'c:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.softwarecompany'
Enter Employee Id: vw4234545
Vendor has no monthly salary details.
PS C:\Users\Admin\OneDrive\Documents\basicjava> c:: cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'c:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.softwarecompany'
Enter Employee Id: wr443256
Invalid Employee Id.
```

✦ Result

The Employee Payroll System was successfully implemented in Java using the concept of interfaces. The `Employee` interface was defined with the `salaryDetails()` method, and `RegularEmployee`, `ContractEmployee`, and `Vendor` classes implemented it, each providing their own salary calculation logic. The program correctly identifies the employee category from the entered Employee ID and, using an interface reference, invokes the appropriate implementation at runtime — displaying Basic Pay, HRA, T.A, and Total Amount for Regular and Contract employees, and a no-salary message for Vendors.